

Networking From Zero

One switch at a time, up to designing a real cloud network

Starts from	A single bit. Nothing is assumed.
Ends at	Sizing a subnet correctly for a cloud platform
Method	Every idea is built on the one before it. Do not skip ahead.
Contains	14 worked examples, 7 diagrams, 18 practice questions with solutions
Works for	Both the AWS batch and the Azure batch

The order we will follow

1	The bit	A switch that is on or off
2	The octet	Eight switches, and why 255 is the ceiling
3	Writing binary	The box method, step by step
4	The IP address	Which colony, which house
5	The subnet mask	Where the colony ends
6	CIDR	The short way of writing the mask
7	Block size	How far apart networks start
8	Network and broadcast	The three-step method
9	Why it matters	What these values are actually used for
10	Cloud sizing	The five addresses the platform takes

How to read this handout

Keep a pen next to you. Every worked example in this document should be copied out by hand at least once. Reading subnetting and doing subnetting are two completely different activities, and only one of them works.

STEP 1

1. What a Bit Is

A bit is a switch. It has only two positions: **off**, which we write as 0, and **on**, which we write as 1. There is nothing in between.

The reason is simple. Inside a computer there is electricity. Either current is flowing or it is not. There is no halfway state, so there is no way to store a 5 or a 7 directly. A computer can only store off and on.

So how does a computer hold a number like 200? By using **several switches together**.

One switch

A single switch can represent only two values.

```
Switch off  ->  0
Switch on   ->  1
```

Two switches

Put two switches side by side and count the possible combinations.

Switch 2	Switch 1	Number
0	0	0
0	1	1
1	0	2
1	1	3

Four combinations now, from 0 to 3. That is 2×2 .

Three switches

S3	S2	S1	Number	S3	S2	S1	Number
0	0	0	0	1	0	0	4
0	0	1	1	1	0	1	5
0	1	0	2	1	1	0	6
0	1	1	3	1	1	1	7

Eight combinations, from 0 to 7. That is $2 \times 2 \times 2$.

The pattern

Number of switches	Combinations	Written as
1	2	2 to the power 1
2	4	2 to the power 2
3	8	2 to the power 3
4	16	2 to the power 4
8	256	2 to the power 8

Every switch you add **doubles** the number of possibilities. This is because every existing combination can now happen twice - once with the new switch off, and once with it on.

Remember this

Eight switches give 256 different values, from 0 to 255. That single fact explains almost everything that follows in this handout.

STEP 2

2. The Octet, and Why Each Switch Has a Value

Eight switches grouped together are called an **octet**. An IP address is made of four octets, which is why it is written as four numbers separated by dots.

Each of the eight switches has a fixed value. Before looking at those values, it helps to look at ordinary decimal numbers, because the idea is identical.

How ordinary numbers already work

When you write 347, you are not writing three digits. You are writing three positions, each with a fixed value.

3	4	7	
hundreds	tens	ones	
100	10	1	
$3 \times 100 + 4 \times 10 + 7 \times 1 = 347$			

Each position is **ten times** the one to its right. That is because each position can hold ten different digits, 0 through 9.

Binary works the same way, with 2 instead of 10

In binary each position can hold only **two** digits, 0 or 1. So each position is **two times** the one to its right.

Start at 1 and keep doubling:

1 -> 2 -> 4 -> 8 -> 16 -> 32 -> 64 -> 128

Write those eight values in a row, largest on the left, and you have the octet.



$$128 + 64 + 8 = 200$$

Eight switches with their fixed values. Here three are on - 128, 64 and 8 - and they add up to 200.

Why 255 is the maximum

Turn every switch on and add the values.

$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

There is no way to write 256 in eight switches, because there is no ninth switch. This is why no part of an IP address can ever be higher than 255. If you type 10.0.1.300 into any cloud portal it will be rejected, and now you know exactly why.

Remember this

Eight switches. Values 128, 64, 32, 16, 8, 4, 2, 1. All on gives 255. That is the whole octet.

STEP 3

3. Writing a Number in Binary

This is a mechanical procedure. It is not something to understand once - it is something to practise until your hand does it without thinking.

Always start by drawing the box

Before doing anything else, write the eight values with eight empty places underneath.

128	64	32	16	8	4	2	1
—	—	—	—	—	—	—	—

There is only one rule

Move from left to right. At each place, ask one question:

The only question you ever ask

"Is the number I have left greater than or equal to this value?"

Yes - write 1, and subtract the value from your number.

No - write 0, and leave your number unchanged.

Worked example 1 - write 150 in binary

Worked example 1

Starting number: 150

128:	is	150	>=	128 ?	YES	->	1	150 - 128 = 22
64:	is	22	>=	64 ?	no	->	0	still 22
32:	is	22	>=	32 ?	no	->	0	still 22
16:	is	22	>=	16 ?	YES	->	1	22 - 16 = 6
8:	is	6	>=	8 ?	no	->	0	still 6
4:	is	6	>=	4 ?	YES	->	1	6 - 4 = 2
2:	is	2	>=	2 ?	YES	->	1	2 - 2 = 0
1:	is	0	>=	1 ?	no	->	0	finished

Result: 150 = 10010110

Check: 128 + 16 + 4 + 2 = 150 correct

Worked example 2 - write 37 in binary

Worked example 2

Starting number: 37

128:	no	-> 0	8:	is 5 >= 8 ?	no	-> 0	
64:	no	-> 0	4:	is 5 >= 4 ?	YES	-> 1	5 - 4 = 1
32:	YES	-> 1	(37-32=5)	2:	is 1 >= 2 ?	no	-> 0
16:	no	-> 0	1:	is 1 >= 1 ?	YES	-> 1	1 - 1 = 0

Result: 37 = 00100101

Check: 32 + 4 + 1 = 37 correct

Two mistakes that cost marks

- **Stopping early.** Once your number reaches 0, the work is done - but the remaining places still need zeros written in them. An octet always has exactly eight digits.
- **Not checking.** Add up the values where you wrote 1. If you get your original number back, you are right. This takes ten seconds and catches almost every error.

Going the other way is much easier

Binary to decimal needs no subtraction at all. Just add the values wherever there is a 1.



$$128 + 64 + 32 = 224$$

11100000 - three switches on, so 128 + 64 + 32 = 224.

10110000	->	128 + 32 + 16	=	176
11000000	->	128 + 64	=	192
10100000	->	128 + 32	=	160
11111000	->	128 + 64 + 32 + 16 + 8	=	248

STEP 4

4. The IP Address

Think about a house address: **Meerut, Sector 4, House 12**.

It has two parts. **Meerut, Sector 4** tells you which colony. **House 12** tells you which house inside that colony. A postman uses the first part to reach the right area, and the second part to reach the right door.

An IP address is built exactly the same way.



10.0.1.7 with a /24. The first three octets say which colony; the last says which house.

An IP address is four octets - thirty-two switches in total.

```
10.0.1.7

10 = 00001010
0  = 00000000
1  = 00000001
7  = 00000111

Together: 00001010 . 00000000 . 00000001 . 00000111
          |--8--|   |--8--|   |--8--|   |--8--|   = 32 bits
```

The problem the address alone cannot solve

Look at 10.0.1.7 on its own. Where does the colony end and the house begin?

It could be colony 10.0.1 with house 7. It could be colony 10.0 with house 1.7. It could be colony 10 with house 0.1.7. **The address by itself does not say.**

So a second piece of information is always needed, and that is the subject of the next section.

Remember this

An IP address has a network part and a host part - which colony, and which house. But the address alone never tells you where the split is.

STEP 5

5. The Subnet Mask

The subnet mask is the second piece of information. Its only job is to say **how many of the 32 bits belong to the network**.

The mask is another 32-bit number. Every place holding a 1 means "this bit is part of the network". Every place holding a 0 means "this bit identifies the machine".

```
IP address    10.0.1.7      00001010 00000000 00000001 00000111
Subnet mask   255.255.255.0 11111111 11111111 11111111 00000000
                                   |----- network -----| |- host -|
```

```
Count the ones in the mask: 24
```

One rule the mask always obeys

In a subnet mask, **all the ones come first and all the zeros come after**. They never mix.

```
11110000    valid    - ones together, then zeros
11111100    valid
10110000    NOT a valid mask - the ones are broken up
```

This single rule is what makes the next part so short.

Because of that rule, only nine numbers can ever appear

If all the ones must come first, then within one octet there are only nine possibilities - zero ones, one one, two ones, and so on up to eight.

Number of ones	Binary	Decimal	Seen in
0	00000000	0	the empty octets of any mask
1	10000000	128	/25
2	11000000	192	/26
3	11100000	224	/27
4	11110000	240	/28
5	11111000	248	/29
6	11111100	252	/30
7	11111110	254	/31
8	11111111	255	the full octets of any mask

Notice the pattern down the decimal column: 128, 192, 224, 240, 248, 252, 254, 255. Each step adds half of the previous addition - 64, then 32, then 16, then 8, 4, 2, 1. If you forget the table you can rebuild it in a few seconds.

Worked example 3 - what is the mask for /26?

Worked example 3

Step 1 Write 26 ones, then fill the rest with zeros.

```
11111111 . 11111111 . 11111111 . 11000000
   8      +   8      +   8      +   2      = 26 ones
```

Step 2 Convert each octet.

```
11111111 = 255      11111111 = 255
11111111 = 255      11000000 = 192
```

Step 3 Write it out. /26 = 255.255.255.192

Worked example 4 - the other direction

Worked example 4

Given the mask 255.255.255.224, what is the prefix?

224 in binary = 11100000 -> three ones

The first three octets are 255, which is 8 ones each = 24

24 + 3 = /27

The interesting octet

In any mask there is one octet that is neither 255 nor 0 - the one where the ones stop partway through. This is called the **interesting octet**, and every calculation in the rest of this handout happens there. The 255s and 0s just come along for the ride.

```
/26 = 255.255.255.192      interesting octet is the 4th, value 192
/20 = 255.255.240.0        interesting octet is the 3rd, value 240
/24 = 255.255.255.0        no interesting octet - it lands exactly on a boundary
```

STEP 6

6. CIDR - The Short Way of Writing the Mask

Writing 255.255.255.0 every time is tedious. Since the mask is just a count of ones, we can write the count instead. That is **CIDR notation**.

```
255.255.255.0    the mask, written the old way
/24              the same thing, written as CIDR

10.0.1.0/24      an address and its prefix together - a CIDR block
```

CIDR stands for Classless Inter-Domain Routing. The name is long, but for your purposes it means one thing: **writing the mask as a number after a slash**.

Where it came from, and why the word classless matters

Networks used to be divided into fixed classes. You could have one of three sizes and nothing in between.

Class	Fixed mask	Addresses you got
Class A	255.0.0.0	16,777,216
Class B	255.255.0.0	65,536
Class C	255.255.255.0	256

Now imagine an organisation that needs 500 addresses. A Class C gives 256 - too few. A Class B gives 65,536 - and roughly 65,000 of them are wasted. There was nothing in between, and the world was running out of addresses because of it.

CIDR removed the classes. Any prefix became allowed - /23, /26, /29, anything. An organisation needing 500 addresses takes a /23, which gives 512. Almost nothing is wasted. That is exactly what **classless** means: the old fixed classes no longer apply.

Remember this

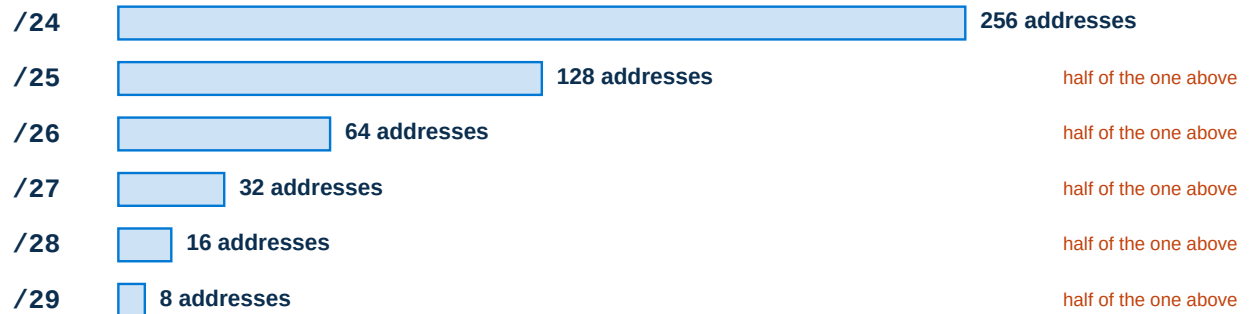
CIDR is not a new topic to learn. It is the same subnet mask, written shorter - and because of it you can build a network of any size instead of only three fixed sizes.

The one calculation you need

The prefix says how many bits the network takes. Everything left over belongs to hosts.

```
host bits = 32 - prefix
addresses = 2 to the power of (host bits)
```

```
/24 -> 32 - 24 = 8 host bits -> 2^8 = 256
/25 -> 32 - 25 = 7 host bits -> 2^7 = 128
/26 -> 32 - 26 = 6 host bits -> 2^6 = 64
/27 -> 32 - 27 = 5 host bits -> 2^5 = 32
/28 -> 32 - 28 = 4 host bits -> 2^4 = 16
/29 -> 32 - 29 = 3 host bits -> 2^3 = 8
```



Each step up in prefix halves the network. A /24 is not slightly bigger than a /28 - it is sixteen times bigger.

The thing that confuses everybody at first

A bigger prefix number means a **smaller** network. The number counts how many bits the network has locked up, so a bigger number leaves fewer bits free for machines. A /29 is tiny; a /16 is enormous.

STEP 7

7. Block Size

Everything so far has been preparation. Block size is the idea that makes subnetting easy.

The formula

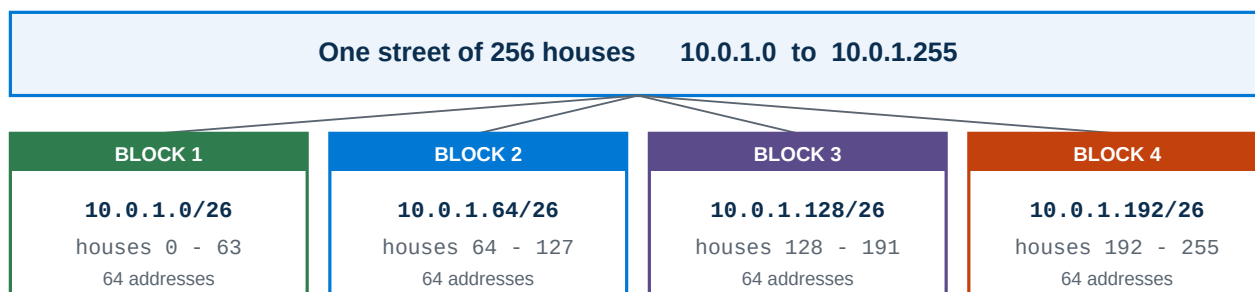
block size = 256 - interesting octet

Prefix	Mask	Interesting octet	Block size
/25	255.255.255.128	128	256 - 128 = 128
/26	255.255.255.192	192	256 - 192 = 64
/27	255.255.255.224	224	256 - 224 = 32
/28	255.255.255.240	240	256 - 240 = 16
/29	255.255.255.248	248	256 - 248 = 8

What block size actually tells you

Go back to the street analogy. A street has 256 houses, numbered 0 to 255. You want to divide that street into smaller blocks.

Block size tells you how many houses are in each block - and therefore where each new block begins.



Jump size 64. Blocks start at 0, 64, 128, 192 - and nowhere else.

A /26 has block size 64, so the street splits into four blocks starting at 0, 64, 128 and 192.

The blocks always start at multiples of the block size. A /26 block can start at 0, 64, 128 or 192 - never at 50, never at 100. This is not a convention someone chose; it falls out of the binary, because those are the only places where the host bits are all zero.

Worked example 5 - list the blocks for a /27

Worked example 5

Block size for /27 = $256 - 224 = 32$

Count in 32s from zero:

0, 32, 64, 96, 128, 160, 192, 224

So the eight possible blocks are:

0 - 31	64 - 95	128 - 159	192 - 223
32 - 63	96 - 127	160 - 191	224 - 255

Remember this

Block size is the jump size. Write the jumps out fully - 0, 64, 128, 192 - never stop halfway. Most mistakes happen because the list was left incomplete.

STEP 8

8. Finding the Network and Broadcast Address

This is the question you will be asked, in exams and in real work: given an address and a prefix, which network is it in, and where does that network end?

Three steps. Always the same three.

- 1 Work out the block size.** 256 minus the interesting octet of the mask.
- 2 Count in block-size jumps from zero** until you pass the address you were given. The last jump you landed on before passing it is the **network address**.
- 3 Take the next jump and subtract one.** That is the **broadcast address**. Everything between the two is the range.

Worked example 6 - 10.0.1.100/26

Worked example 6

```
Step 1  /26 mask is 255.255.255.192
        Block size = 256 - 192 = 64

Step 2  Jumps of 64:  0,  64, 128, 192
        Where does 100 fall?  Between 64 and 128.
        Network address  = 10.0.1.64

Step 3  Next jump is 128, so 128 - 1 = 127
        Broadcast          = 10.0.1.127

Answer  Network    10.0.1.64
        Broadcast   10.0.1.127
        Range       10.0.1.64 to 10.0.1.127
        Total       64 addresses
```

Worked example 7 - 192.168.20.200/26

Worked example 7

```
Step 1  Block size = 256 - 192 = 64

Step 2  Jumps of 64:  0,  64, 128, 192
        200 falls between 192 and 256, so it is in the 192 block.
        Network address  = 192.168.20.192

Step 3  The next jump would be 256, which means the octet has ended.
        256 - 1 = 255
        Broadcast          = 192.168.20.255

Answer  Range  192.168.20.192 to 192.168.20.255  (64 addresses)
```

256 is never an address

When your jumps reach 256 it simply means the octet has run out - remember that eight switches stop at 255. There is no such address as 192.168.20.256. The 256 in your working is a boundary marker, not a place.

Worked example 8 - 172.16.9.35/27

Worked example 8

Step 1 /27 mask is 255.255.255.224. Block size = $256 - 224 = 32$

Step 2 Jumps of 32: 0, 32, 64, 96, 128, 160, 192, 224
 35 falls between 32 and 64.
 Network address = 172.16.9.32

Step 3 Next jump 64, minus one = 63
 Broadcast = 172.16.9.63

Answer Range 172.16.9.32 to 172.16.9.63 (32 addresses)

Worked example 9 - why /24 needs no working at all

A /24 mask is 255.255.255.0. The interesting octet is 0, so the block size is 256 - the whole octet is one single block. There is nothing to divide.

Worked example 9

192.168.1.10/24

Network	192.168.1.0	just set the last octet to 0
Broadcast	192.168.1.255	just set the last octet to 255
Range	192.168.1.0 to 192.168.1.255	
Total	256 addresses	

More examples, read straight off:

10.0.1.55/24	-> network 10.0.1.0	broadcast 10.0.1.255
172.16.9.200/24	-> network 172.16.9.0	broadcast 172.16.9.255

This is one reason real organisations use /24 so often. There is no arithmetic to get wrong, so there are fewer mistakes. The jump-counting work is only needed for /25 through /29, where the octet gets split.

Worked example 10 - are two addresses in the same network?

Worked example 10

Question: 10.0.1.20/26 and 10.0.1.80/26 - same network?

Block size 64. Blocks: 0-63, 64-127, 128-191, 192-255

10.0.1.20	-> falls in 0-63	-> network 10.0.1.0
10.0.1.80	-> falls in 64-127	-> network 10.0.1.64

Answer: different networks.

Consequence: they cannot talk to each other directly. Traffic must go through the gateway, and a firewall rule could block it.

9. What These Three Values Are Actually For

The network address

This is the **name of the network**. No machine is ever given it. Every time you refer to a network anywhere, this is what you write:

- Creating a subnet in a cloud portal - you type the network address and prefix.
- Writing a firewall rule - "allow traffic from 10.0.1.64/26".
- Adding a route - "send anything for 10.0.1.64/26 this way".

The broadcast address

Traditionally this meant "send to everyone on this network at once". Most cloud platforms do not support broadcast traffic, but the address is **still reserved**, so it still costs you one address.

Its practical use to you is different: it tells you **where the network ends**, and therefore where the next one can begin.

The range

This is the most useful of the three in daily work, for two reasons.

- **Planning.** The range tells you how many machines will fit.
- **Checking for overlap.** Two networks must never claim the same address, and the only way to be sure is to write out both ranges and compare them.

```
Existing subnet    10.0.1.0/24      range 10.0.1.0    to 10.0.1.255
You try to add     10.0.1.128/25   range 10.0.1.128  to 10.0.1.255
```

```
The second range sits entirely inside the first. This will be rejected.
```

10. How Real Organisations Do This

In a real company, nobody opens a portal and invents an address range on the spot. There is a plan first, and the plan is a document.

Everything starts with an address plan

A network team keeps a master list of which range belongs to which part of the business. New projects are **allocated** a range from it. Nobody chooses their own.

Range	Used by	Where
10.0.0.0/16	Production	Cloud, primary region
10.1.0.0/16	Development and test	Cloud, primary region
10.2.0.0/16	Disaster recovery	Cloud, second region
10.10.0.0/16	Head office	On-premise
10.11.0.0/16	Branch offices	On-premise
10.50.0.0/16	VPN and remote users	-

The reason is simple and expensive to learn the hard way. Today these are separate networks. Next year somebody will want to connect two of them with a VPN or a peering link. If their ranges overlap, that connection cannot be made, and the only fix is to rebuild one of the networks from scratch. In production, that is not a realistic option.

The question asked in every real connectivity project

"What address range are you using?" This is asked before any technical work begins. If both sides answer with overlapping ranges, the project stops there until one side rennumbers.

They take more space than they need

A beginner sizing a subnet for 40 servers picks a /26, because 64 is enough. An experienced engineer picks a /24 and takes 256.

The reason is that resizing a subnet after it is in use is painful - it usually requires the subnet to be emptied first, which means downtime. Private address space, on the other hand, costs nothing. There are over sixteen million addresses in 10.0.0.0/8. Taking four times what you need today is cheap insurance.

They separate tiers into different subnets - for security

This is the most important practical habit in the list.

```
Network    10.0.0.0/16

10.0.1.0/24    web tier        reachable from the internet
10.0.2.0/24    application tier reachable only from the web tier
10.0.3.0/27    database tier   reachable only from the application tier
10.0.4.0/28    management     reachable only from the office
```

All of these could have gone in one large subnet. They are deliberately separated, because **firewall rules are applied to subnets**. Only when the database sits in its own subnet can you write a rule that says "accept database traffic from the application tier and from nowhere else".

The subnet boundary is the security boundary. This is also what auditors look for - a database that shares a subnet with internet-facing servers is a finding in any serious security review.

They leave deliberate gaps

Look at the plan above. The database tier is a /27, which ends at 10.0.3.31. The next entry starts at 10.0.4.0, leaving everything from 10.0.3.32 upward unused.

That gap is intentional. If the database tier ever needs to grow, the space beside it is free. Without the gap, growing it would mean moving it somewhere else entirely.

STEP 10

11. Sizing a Subnet in the Cloud

In ordinary networking, two addresses in every network are unavailable: the network address and the broadcast address. A /24 therefore gives 254 usable addresses. That is the standard answer and it is correct - outside the cloud.

Cloud platforms reserve more. On Azure, five addresses are taken in every subnet.

Subnet 10.0.1.0/24 256 addresses in total



Five addresses are taken by Azure in EVERY subnet, whatever its size.

Ordinary networking takes 2 (network id and broadcast). Azure takes 3 more: gateway and two DNS.

Address	Reserved for	Familiar?
first address	Network identifier	Yes - ordinary networking takes this too
second address	Default gateway	New - the router the subnet sends traffic to
third address	Platform DNS	New
fourth address	Platform DNS	New
last address	Broadcast	Yes - ordinary networking takes this too

What this does to each size

Prefix	Host bits	Total addresses	Minus 5	Machines you can place
/24	8	256	256 - 5	251
/25	7	128	128 - 5	123
/26	6	64	64 - 5	59
/27	5	32	32 - 5	27
/28	4	16	16 - 5	11
/29	3	8	8 - 5	3

Notice how severe this is on small subnets. A /29 has eight addresses and five are taken, leaving three. This is why /29 is the smallest subnet the platform allows - anything smaller would leave nothing at all.

The sizing rule

- 1 Start with the number of machines the subnet must hold.
- 2 **Add five.** This is the step almost everyone forgets.
- 3 Choose the smallest prefix whose total is equal to or greater than that number.

Worked example 11 - 12 machines

Worked example 11

```
12 machines + 5 reserved = 17 addresses needed
```

```
/28 gives 16    -> not enough
```

```
/27 gives 32    -> fits
```

```
Answer: /27
```

Worked example 12 - the classic trap

Worked example 12

```
60 machines needed.
```

```
The instinct is to say /26, because a /26 has 64 addresses  
and 60 is less than 64.
```

```
But: 60 + 5 = 65 addresses needed
```

```
/26 gives 64    -> short by one
```

```
/25 gives 128   -> fits
```

```
Answer: /25
```

Add the five before you choose, not after

Every question of this kind is testing whether you remembered the reserved addresses. If the requirement lands just below a power of two - 12, 27, 60 - it is almost certainly deliberate.

12. Practice

Work these with a pen. Solutions follow on the next page, so cover them.

Set A - binary

- | | |
|-------------------------|----------------------------------|
| A1) Write 64 in binary | A4) What is 11100000 in decimal? |
| A2) Write 150 in binary | A5) What is 10100000 in decimal? |
| A3) Write 37 in binary | A6) What is 11111100 in decimal? |

Set B - masks and CIDR

- B1) What is the subnet mask for /27?
- B2) What is the subnet mask for /28?
- B3) The mask is 255.255.255.224. What is the prefix?
- B4) The mask is 255.255.255.128. What is the prefix?
- B5) How many total addresses does a /26 have?

Set C - network and broadcast

- | | |
|--------------------|-----------------------|
| C1) 10.0.5.130/25 | C3) 192.168.20.200/26 |
| C2) 172.16.9.35/27 | C4) 10.1.1.250/28 |

Set D - cloud sizing

- D1) A subnet must hold 25 machines. Smallest prefix?
- D2) A subnet must hold 50 machines. Smallest prefix?
- D3) How many machines fit in a /27?
- D4) How many machines fit in a /28?
- D5) Why is /29 the smallest subnet allowed?

13. Solutions

Set A

```
A1) 64 = 01000000      A4) 11100000 = 128+64+32 = 224
A2) 150 = 10010110     A5) 10100000 = 128+32 = 160
A3) 37 = 00100101      A6) 11111100 = 128+64+32+16+8+4 = 252
```

Set B

```
B1) /27 = 255.255.255.224 (27 ones: 8+8+8+3, and 11100000 = 224)
B2) /28 = 255.255.255.240 (28 ones: 8+8+8+4, and 11110000 = 240)
B3) 224 = 11100000 -> 3 ones. 24 + 3 = /27
B4) 128 = 10000000 -> 1 one. 24 + 1 = /25
B5) /26 -> 32 - 26 = 6 host bits -> 2^6 = 64 addresses
```

Set C

```
C1) 10.0.5.130/25
Block size 128. Jumps: 0, 128. 130 falls in the 128 block.
Network 10.0.5.128 Broadcast 10.0.5.255 128 addresses

C2) 172.16.9.35/27
Block size 32. Jumps: 0, 32, 64... 35 falls in the 32 block.
Network 172.16.9.32 Broadcast 172.16.9.63 32 addresses

C3) 192.168.20.200/26
Block size 64. Jumps: 0, 64, 128, 192. 200 falls in the 192 block.
Network 192.168.20.192 Broadcast 192.168.20.255 64 addresses

C4) 10.1.1.250/28
Block size 16. Jumps: ... 224, 240, (256). 250 falls in the 240 block.
Network 10.1.1.240 Broadcast 10.1.1.255 16 addresses
```

Set D

```
D1) 25 + 5 = 30 needed. /28 gives 16 (too small), /27 gives 32. Answer /27
D2) 50 + 5 = 55 needed. /27 gives 32 (too small), /26 gives 64. Answer /26
D3) /27 has 32 addresses. 32 - 5 = 27 machines
D4) /28 has 16 addresses. 16 - 5 = 11 machines
D5) A /29 has 8 addresses and 5 are reserved, leaving 3. Anything smaller
would have 4 addresses or fewer, so nothing would be left.
```

14. One-Page Summary

Print this page and keep it with you.

The octet

```
128  64  32  16   8   4   2   1           all on = 255
```

```
To write a number: left to right, ask 'does this value fit?'  
Yes -> write 1 and subtract.  No -> write 0.
```

Mask octet values - the only nine that exist

```
0      128    192    224    240    248    252    254    255  
      /25     /26     /27     /28     /29     /30     /31
```

Sizes

Prefix	Mask	Block size	Total	Usable in cloud
/24	255.255.255.0	256	256	251
/25	255.255.255.128	128	128	123
/26	255.255.255.192	64	64	59
/27	255.255.255.224	32	32	27
/28	255.255.255.240	16	16	11
/29	255.255.255.248	8	8	3

The three-step method

1. Block size = 256 - interesting octet
2. Count in block-size jumps from 0 until you pass the address.
The jump you landed on is the network address.
3. Next jump minus one = broadcast address.

Write the jumps out fully. 256 is never an address.

Cloud sizing

Five addresses are reserved in every subnet:
network id, gateway, two DNS, broadcast

Rule: machines needed + 5 -> choose the prefix that fits

Private ranges

```
10.0.0.0/8      10.0.0.0      to 10.255.255.255  
172.16.0.0/12   172.16.0.0     to 172.31.255.255  
192.168.0.0/16  192.168.0.0    to 192.168.255.255
```

A network is an address range you reserved. A subnet is a slice of it. Everything else is rules about what may cross those boundaries.